AI Agent Mimarisi

Teknik Dokümantasyon

Laravel + Anthropic Claude ile tool-use tabanlı müşteri hizmetleri asistanı: mimari, veri akışı, tasarım kararları ve kaynak kodun tamamı

Proje	exampleai (Laravel 13, PHP 8.3)

Kapsam	`app/Http/Controllers/Ai/AgentController.php`
	`app/Services/Ai/AiAgent.php`
	`app/Services/Ai/AiClient.php`
	`app/Services/Ai/ToolsService.php`
	`app/Schema/ToolSchema.php`

Bağlam olarak incelenenler	routes/web.php, config/services.php, app/Models/Customer.php, app/Models/Order.php, customers/orders migration'ları, resources/views/ai/agent.blade.php, resources/views/components/ai/chat.blade.php
Hedef okuyucu	Projeye aylar sonra dönen veya projeye yeni katılan geliştirici
Tarih	12 Ekim 2026

Not: Bu dokümandaki tüm kod alıntıları ve tam kaynak kodları, PDF üretildiği anda doğrudan proje dosyalarından okunmuştur; satır numaraları gerçek dosyalarla birebir aynıdır.

İçindekiler

1. Genel Mimari 4

1.1 Sistem ne yapıyor? 4

1.2 Katmanlar ve sorumluluklar 4

1.3 Temel tasarım kararları 4

1.4 Bileşenler nasıl haberleşiyor? 5

1.5 Anthropic tool-use protokolü (kısa özet) 5

2. Flowchart (Akış Şeması) 6

2.1 Uçtan uca istek akışı 7

2.2 Sınıf / bağımlılık diyagramı 8

2.3 Döngü boyunca $messages dizisinin büyümesi 9

3. Veri Akışı 10

3.1 Adım adım istek yolculuğu 10

3.2 Somut örnek: tek tool çağrılı soru 10

3.3 Paralel / çok adımlı tool kullanımı 12

3.4 Hata akışları 12

3.5 Süre ve maliyet profili 12

4. Klasör Yapısı 13

4.1 İlgili dizin ağacı 13

4.2 Her klasörün görevi 13

5. Dosya Açıklamaları 15

5.1 AgentController 15

5.2 AiAgent 15

5.3 AiClient 16

5.4 ToolsService 16

5.5 ToolSchema 17

6. Kod Analizleri 18

6.1 AgentController.php 18

6.2 AiAgent.php 19

6.3 AiClient.php 22

6.4 ToolsService.php 23

6.5 ToolSchema.php 26

6.6 Çapraz konular ve tasarım değerlendirmesi 27

7. Dosyaların Tam Kaynak Kodları 30

7.1 AgentController.php 30

7.2 AiAgent.php 31

7.3 AiClient.php 32

7.4 ToolsService.php 33

7.5 ToolSchema.php 34

7.6 Akışa dahil destekleyici dosyalar 35

8. Genel Mimari Özeti 37

8.1 Tek paragrafta sistem 37

8.2 Her dosya tek cümlede 37

8.3 Akılda tutulacak sözleşmeler 37

8.4 Yeni bir tool nasıl eklenir? (reçete) 37

8.5 İyileştirme yol haritası 38

8.6 Altı ay sonra geri döndüğünüzde kontrol listesi 38

1. Genel Mimari

1.1 Sistem ne yapıyor?

Bu modül, bir e-ticaret mağazası için Türkçe konuşan bir **müşteri hizmetleri asistanıdır**. Kullanıcı tarayıcıdaki sohbet kutusuna doğal dilde bir soru yazar (ör. “ORD-1003 siparişinin durumu ne?” veya “Ayşe Demir toplam ne kadar harcamış?”). Soru Laravel üzerinden Anthropic'in Claude modeline iletilir.

Model soruyu cevaplamak için veritabanına bakması gerektiğine karar verirse **kendisi SQL çalıştırmaz**. Bunun yerine, uygulamanın önceden tanımladığı iki **tool**'dan (araç) birini belirli parametrelerle çağırmak istediğini bildirir. Laravel bu isteği güvenli şekilde yerel bir PHP metoduna çevirir, Eloquent ile veriyi okur, sonucu modele geri yollar. Model de son cevabı bu gerçek veriye dayanarak üretir.

Bu desene **tool use (function calling) ile çalışan ajan döngüsü** (agent loop) denir. İş bölümü şöyledir: *ne* yapılacağına model karar verir, *nasıl* ve *hangi sınırlar içinde* yapılacağını uygulama belirler. Böylece model veriye yalnızca uygulamanın açtığı dar kapılardan erişebilir.

> **Zihinsel model**
> Ajan = LLM + araçlar + döngü. Model bir “beyin” gibidir ama veritabanına dokunamaz. ToolsService onun “elleri”, ToolSchema ellerin yapabileceği işlerin “menüsü”, AiAgent koordinatör, AiClient ise dış dünyaya açılan “telefon hattı”dır. AgentController de bu düzeneği HTTP dünyasına bağlayan kapıdır.

1.2 Katmanlar ve sorumluluklar

Katman	Dosya(lar)	Sorumluluk
Arayüz (UI)	resources/views/ai/agent.blade.php, components/ai/chat.blade.php	Sohbet ekranı; `fetch` ile JSON POST atar, `reply` alanını ekrana basar. (Kapsam dışı ama akışın başlangıcı.)
Yönlendirme	routes/web.php	`GET /ai/agent` → sayfa, `POST /ai/agent/send` → mesaj uç noktası.
HTTP / Controller	app/Http/Controllers/Ai/AgentController.php	Girdiyi doğrular, ajanı çalıştırır, hataları HTTP durum kodlarına çevirir, JSON döner.
Orkestrasyon (Ajan)	app/Services/Ai/AiAgent.php	System prompt, tool listesi, çok adımlı döngü, tool sonuçlarını modele geri besleme, adım sınırı.
API istemcisi	app/Services/Ai/AiClient.php	Anthropic Messages API'ye tek bir HTTP çağrısı (URL, header, model, timeout).
Tool yürütme	app/Services/Ai/ToolsService.php	Tool adını güvenle PHP metoduna eşler, Eloquent sorgularını çalıştırır, sonucu formatlar.
Sözleşme (Schema)	app/Schema/ToolSchema.php	Modele gösterilen tool tanımları: ad, açıklama, JSON Schema. Aynı zamanda whitelist kaynağı.
Veri	app/Models/Customer.php, Order.php	Eloquent modelleri; `Customer hasMany Order`, `Order belongsTo Customer`.
Yapılandırma	config/services.php, .env	`services.anthropic.key` ve `services.anthropic.model` değerleri.

1.3 Temel tasarım kararları

- **Sorumlulukların ayrılması (Separation of Concerns).** HTTP işleri (Controller), iş akışı (AiAgent), dış servis iletişimi (AiClient), iş verisi (ToolsService) ve sözleşme (ToolSchema) ayrı dosyalardadır. Her biri diğerlerini bozmadan değiştirilebilir ve ayrı ayrı test edilebilir.

- **SDK yerine ham HTTP.** Resmî Anthropic SDK yerine Laravel'in `Http` facade'ı kullanılmıştır. Ek paket bağımlılığı yoktur, `Http::fake()` ile test edilebilir; timeout, retry, log gibi özellikler hazırdır.
- **Anthropic formatı sistemin ortak dilidir.** Katmanlar arasında özel DTO sınıfları yoktur; veri, Messages API'nin kendi JSON yapısıyla (PHP dizileri) taşınır: `$messages`, `content` blokları, `stop_reason`. Dönüştürme kodu gerekmez. Bedeli: tip güvenliği düşüktür, dizi anahtarlarını bilmek gerekir.
- **Tool tanımı ile tool uygulaması ayrıdır.** Modelin gördüğü tanım `ToolSchema`'da, çalışan kod `ToolsService`'tedir. İkisi tool adı üzerinden eşleşir: **tool adı = metod adı**.
- **Whitelist ile güvenli dinamik çağrı.** Model yalnızca `ToolSchema`'da listelenen adları çağırabilir; başka bir public metodu (ör. `call` veya `formatOrder`) tetikleyemez.
- **Sınırlı döngü.** `maxSteps = 5`; model sonsuz tool çağrısı zincirine giremez. Maliyet ve süre sınırlıdır.
- **Durumsuz (stateless) sohbet.** Her HTTP isteği yalnızca son kullanıcı mesajıyla başlar; önceki mesajlar sunucuya gönderilmez, saklanmaz (chat bileşenindeki yorum: “geçmiş tutulmaz”). Basitlik kazanılır; ancak “peki onun diğer siparişi?” gibi bağlama dayalı takip soruları çalışmaz.
- **Tool hataları exception değil, veridir.** Tool'lar `['status' => 'error', 'message' => …]` döner. Bu sonuç modele `is_error: true` ile iletilir ve model kullanıcıya doğal bir dille “bulunamadı” der. Sadece altyapı hataları (API, ağ) exception olarak yukarı çıkar.

1.4 Bileşenler nasıl haberleşiyor?

Kaynak → Hedef	Mekanizma	Taşınan veri
chat.blade → AgentController	HTTP POST (JSON) + CSRF header	`{message: string}`
Laravel → AgentController::post	Route + method injection	`Request`, `AiAgent` örneği
AgentController → AiAgent	`run(array $messages)`	`[['role' => 'user', 'content' => mesaj]]`
AiAgent → AiClient	Constructor injection, `send()`	messages, system prompt, tools
AiClient → Anthropic API	HTTPS POST	JSON payload → JSON yanıt (dizi)
AiAgent → ToolSchema	Statik `ToolSchema::all()`	Tool tanımları dizisi
AiAgent → ToolsService	Constructor injection, `call(name, input)`	Tool adı + girdi → sonuç dizisi
ToolsService → ToolSchema	Statik `ToolSchema::all()`	Whitelist için tool adları
ToolsService → Customer/Order	Eloquent sorguları	Model koleksiyonları
AgentController → AiAgent::text	Statik yardımcı	Son yanıt → düz metin
AgentController → chat.blade	JSON yanıt	`{reply}` (200) veya `{message}` (422/502/500)

Dikkat edilirse **hiçbir sınıf bir diğerini** `new` **ile oluşturmaz**. Nesne grafiği (AiAgent → AiClient + ToolsService) tamamen Laravel Service Container tarafından kurulur. Statik çağrılar yalnızca durum taşımayan, saf veri/fonksiyon niteliğindeki yerlerde kullanılmıştır (`ToolSchema::all()`, `AiAgent::text()`).

1.5 Anthropic tool-use protokolü (kısa özet)

Mimarinin neden bu şekilde kurulduğunu anlamak için API'nin tool-use sözleşmesini bilmek gerekir:

1. İstemci `messages`, `system` ve `tools` (her biri ad + açıklama + `input_schema`) ile istek atar.
2. Model bir tool kullanmak isterse yanıtın `stop_reason` değeri `tool_use` olur ve `content` dizisinde bir veya birden fazla `{type: 'tool_use', id, name, input}` bloğu bulunur.
3. İstemci bu tool'ları kendisi çalıştırır. Modelin yanıtını **olduğu gibi** `assistant` mesajı olarak geçmişe ekler, ardından **tek bir** `user` mesajı içinde her tool için `{type: 'tool_result', tool_use_id, content,`

`is_error}` bloğu gönderir.

4. Model artık veriye sahiptir. Ya yeni bir tool ister (döngü sürer) ya da `stop_reason: end_turn` ile nihai metni üretir.

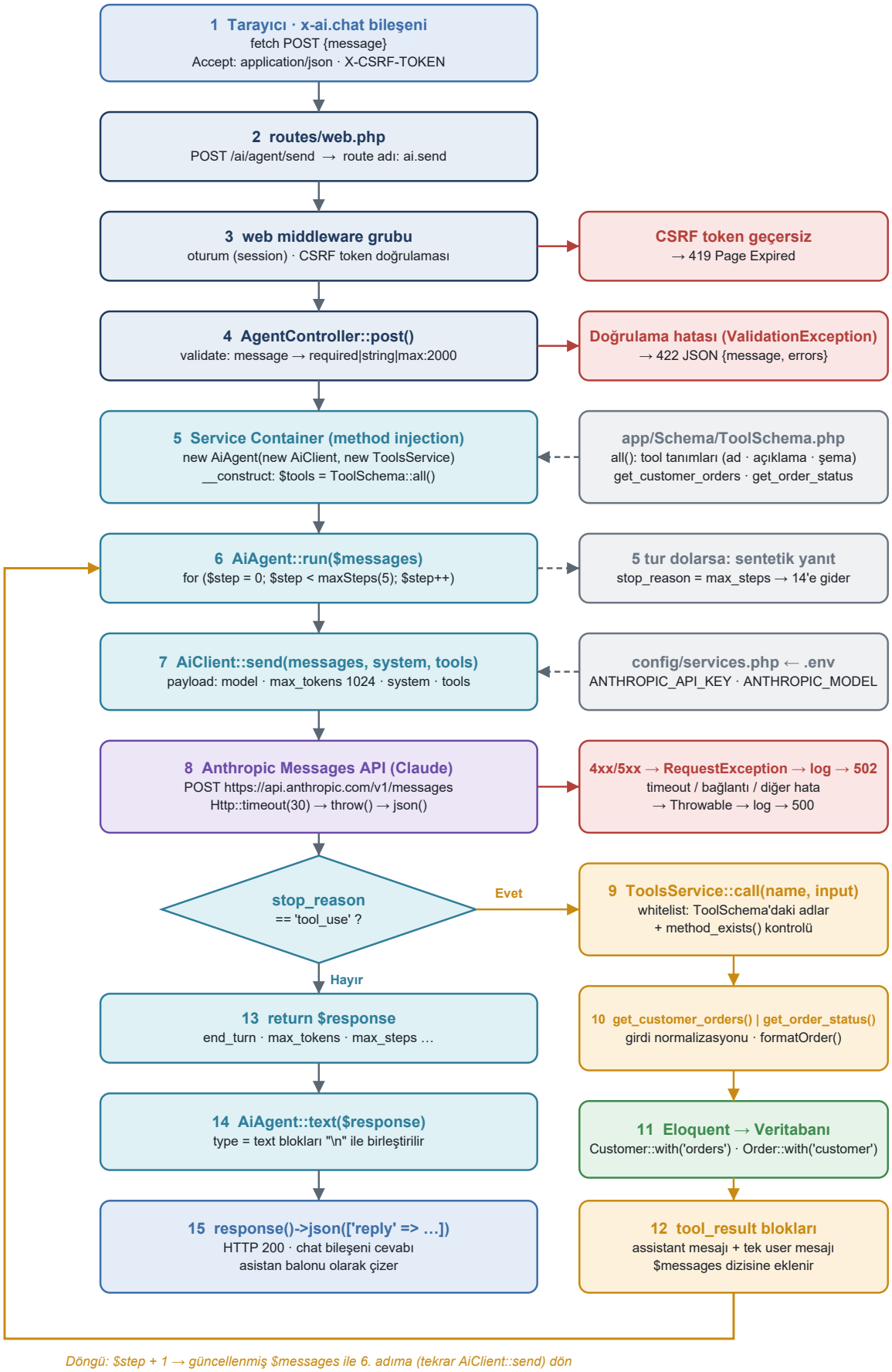
`AiAgent::run()` metodu bu dört adımın birebir PHP karşılığıdır.

2. Flowchart (Akış Şeması)

Aşağıdaki şema, kullanıcının mesajı göndermesinden cevabın ekrana basılmasına kadar geçen **uçtan uca** akışı gösterir. Numaralar "3. Veri Akışı" bölümündeki adım numaralarıyla aynıdır. Sol kolon ana (mutlu) yolu, sağ üst kolon hata çıkışlarını ve yapılandırma/şema girdilerini, sağ alt kolon ise tool yürütme döngüsünü gösterir.

> **Şema nasıl okunur?**
> Kırmızı kutular hata çıkışlarıdır. Gri kesikli oklar yapılandırma veya şema girdisini, turuncu oklar tool döngüsünü gösterir. Turuncu dönüş oku, tool sonuçlarının eklendiği `$messages` dizisiyle bir sonraki tura geçildiğini anlatır.

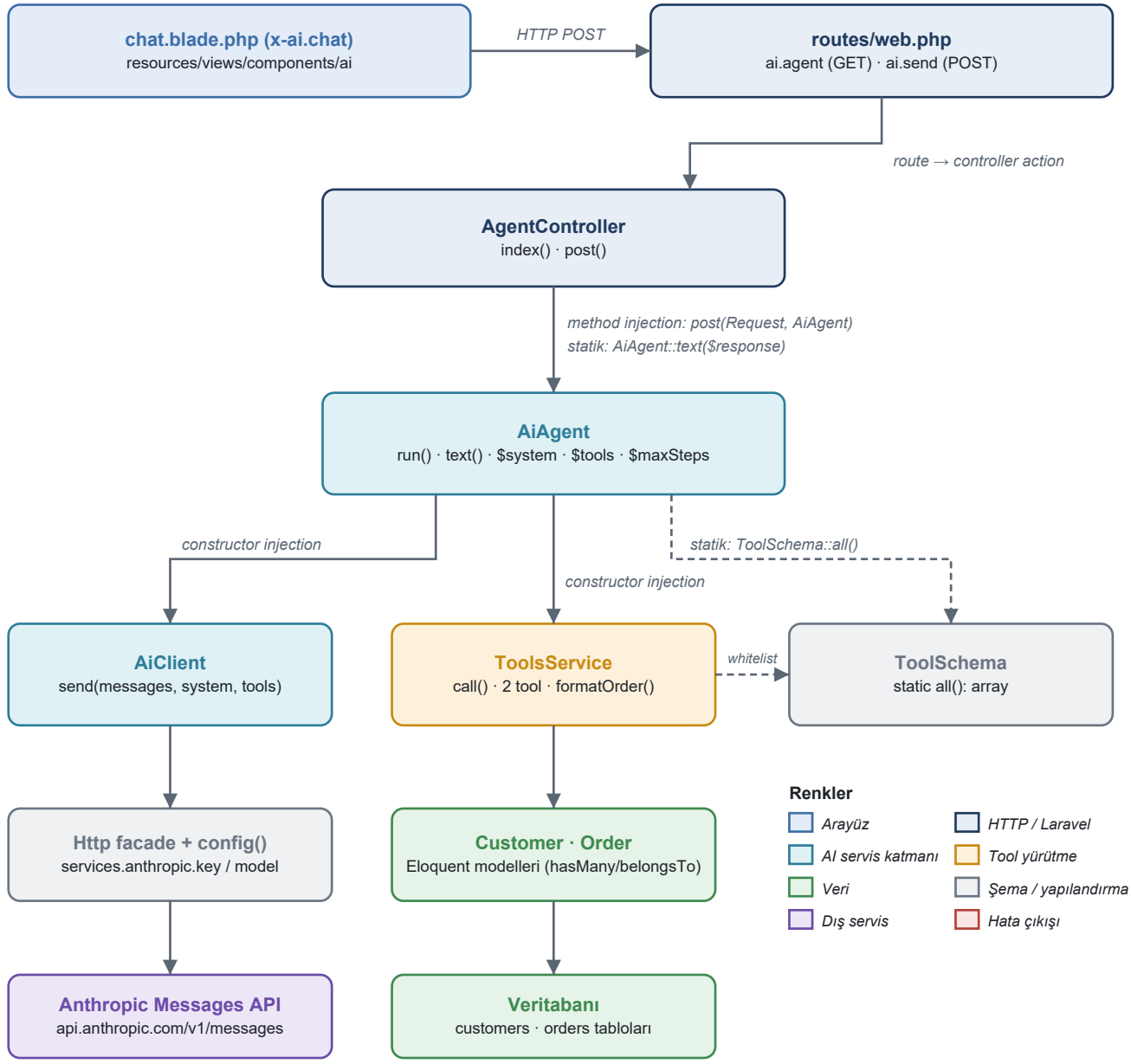
2.1 Uçtan uca istek akışı



Döngü: $step + 1 → güncellenmiş $messages ile 6. adıma (tekrar AiClient::send) dön

2.2 Sınıf / bağımlılık diyagramı

Bu şema “kim kimi kullanıyor” sorusunu cevaplar. Ok yönü bağımlılık yönüdür: ok, kullanan sınıftan kullanılan sınıfa doğrudur. Düz oklar Service Container üzerinden enjekte edilen bağımlılıkları, kesikli oklar statik çağrıları gösterir.



Diyagramdan çıkan önemli sonuçlar:

- **Bağımlılıklar tek yönlüdür ve döngü yoktur.** Controller → Ajan → (İstemci, Tool servisi) → (HTTP, Model). Alt katmanlar üst katmanları tanımaz. AiClient tool'lardan, ToolsService ise LLM'den habersizdir.
- **ToolSchema bir yapraktır.** Hiçbir şeye bağımlı değildir ama iki sınıf tarafından kullanılır. Tool listesinin “tek doğruluk kaynağı” (single source of truth) budur.
- **AiClient Anthropic'i tanıyan tek sınıftır.** Model sağlayıcısı değişirse etkilenen yer çoğunlukla burasıdır (ancak mesaj formatı Anthropic'e özgü olduğu için AiAgent de etkilenir; bkz. 6.6).

2.3 Döngü boyunca $messages dizisinin büyümesi

Ajanın tüm durumu tek bir PHP dizisinde, `$messages` içinde tutulur. Her turda bu dizi büyür ve **tamamı** yeniden API'ye gönderilir (API durumsuzdur; modelin hafızası bu dizidir).

```
TUR 1  gönderilen  : [ user("ORD-1003 siparişinin durumu ne?") ]
       gelen yanıt : stop_reason = tool_use
                     content = [ tool_use(id=toolu_A, name=get_order_status,
                                          input={order_number:"ORD-1003"}) ]

       -> ToolsService::call("get_order_status", {order_number:"ORD-1003"})

TUR 2  gönderilen  : [ user("ORD-1003 siparişinin durumu ne?"),
                       assistant([ tool_use(id=toolu_A, ...) ]),          <- yanıt aynen eklendi
                       user([ tool_result(tool_use_id=toolu_A,
                                          content='{"status":"success",...}') ]) ]
       gelen yanıt : stop_reason = end_turn
                     content = [ text("ORD-1003 numaralı siparişiniz hazırlanıyor ...") ]

       -> return $response  ->  AiAgent::text()  ->  {"reply": "..."}
```

3. Veri Akışı

3.1 Adım adım istek yolculuğu

Numaralar flowchart'taki kutularla eşleşir.

1. **Tarayıcı.** `x-ai.chat` bileşeni, kullanıcı formu gönderdiğinde `fetch(endpoint, {method: 'POST'})` çağırır. Gövde `{"message": "…"}` şeklindedir. Header'lar: `Content-Type: application/json`, `Accept: application/json` ve meta etiketinden okunan `X-CSRF-TOKEN`. `endpoint`, Blade'de `route('ai.send')` ile üretilir.
2. **Route.** `routes/web.php` içindeki `Route::post('/ai/agent/send', [AgentController::class, 'post'])` isteği controller action'ına bağlar.
3. **web middleware grubu.** Route `web.php`'de olduğu için oturum ve CSRF korumasından geçer. Token geçersizse Laravel 419 döner; istek controller'a hiç ulaşmaz.
4. **Doğrulama.** `AgentController::post()` önce `message` alanını doğrular. Hata varsa Laravel bir `ValidationException` fırlatır. İstek `Accept: application/json` taşıdığı için bu otomatik olarak **422** JSON'a (`{message, errors}`) çevrilir; chat bileşeni `error.message` alanını gösterir.
5. **Bağımlılık çözümü.** Laravel, `post()` imzasındaki `AiAgent` tipini görür ve container ile oluşturur: önce `AiClient` ve `ToolsService`'i (ikisinin de constructor bağımlılığı yoktur), sonra bunlarla `AiAgent`'ı. `AiAgent::__construct` tool tanımlarını `ToolSchema::all()` ile bir kez yükler.
6. **Ajan döngüsü başlar.** Controller `run([['role' => 'user', 'content' => $message]])` çağırır. `for` döngüsü en fazla 5 tur döner.
7. **API isteği hazırlanır.** `AiClient::send()` payload'u kurar: `model` (config), `messages`, `max_tokens: 1024`, `system` (persona) ve `tools` (şemalar).
8. **Anthropic çağrısı.** `Http::withHeaders([...])->timeout(30)->post(...)->throw()->json()`. 4xx/5xx yanıtlar `RequestException`'a dönüşür (controller → 502). Timeout ve bağlantı hataları `ConnectionException` fırlatır (controller → `Throwable` → 500).
9. **Tool yürütme.** `stop_reason === 'tool_use'` ise modelin yanıtı `assistant` mesajı olarak eklenir. Her `tool_use` bloğu için `ToolsService::call($name, $input)` çağrılır. `call()` adı whitelist'e karşı kontrol eder.
10. **İş mantığı.** İlgili metot (`get_customer_orders` / `get_order_status`) girdiyi normalize eder.
11. **Veritabanı.** Eloquent sorgusu eager loading ile çalışır. Sonuç `formatOrder()` ile sade bir diziye dönüştürülür.
12. **Sonuçların geri beslenmesi.** Her sonuç JSON string'e çevrilip `tool_result` bloğu yapılır. Tüm bloklar **tek bir** `user` mesajında `$messages`'a eklenir. Döngü 6. adıma döner.
13. **Bitiş.** `stop_reason` artık `tool_use` değilse (tipik olarak `end_turn`) ham yanıt geri döner. 5 tur dolarsa ajan, aynı formatta sentetik bir yanıt üretir (`stop_reason: max_steps`).
14. **Metne çevirme.** Controller `AiAgent::text($response)` ile tüm `text` bloklarını birleştirir. Sonuç boşsa “Bir cevap üretilemedi.” kullanılır.
15. **Yanıt.** `{"reply": "…"}` JSON'u 200 ile döner. Bileşenin `extractReply()` fonksiyonu `reply` alanını okuyup asistan balonu olarak çizer.

3.2 Somut örnek: tek tool çağrılı soru

> **Not**
> Aşağıdaki JSON'larda yapı gerçektir; id, tutar, tarih ve isim gibi değerler örnek amaçlıdır.

(a) Tarayıcı → Laravel

```
POST /ai/agent/send
Content-Type: application/json
Accept: application/json
X-CSRF-TOKEN: 3kV...

{"message": "ORD-1003 siparişinin durumu ne?"}
```

(b) AiClient → Anthropic (1. tur payload'u, kısaltılmış)

```
{
  "model": "<ANTHROPIC_MODEL>",
  "max_tokens": 1024,
  "system": "Sen bir e-ticaret mağazasının müşteri hizmetleri asistanısın. ...",
  "tools": [
    {"name": "get_customer_orders", "description": "...", "input_schema": {...}},
    {"name": "get_order_status",    "description": "...", "input_schema": {...}}
  ],
  "messages": [
    {"role": "user", "content": "ORD-1003 siparişinin durumu ne?"}
  ]
}
```

(c) Anthropic → AiClient (1. tur yanıtı)

```
{
  "id": "msg_01...",
  "role": "assistant",
  "stop_reason": "tool_use",
  "content": [
    {"type": "tool_use", "id": "toolu_01A", "name": "get_order_status",
     "input": {"order_number": "ORD-1003"}}
  ],
  "usage": {"input_tokens": 912, "output_tokens": 58}
}
```

(d) ToolsService::call('get_order_status', …) dönüşü

```
[
  'status' => 'success',
  'order'  => [
      'order_number' => 'ORD-1003',
      'order_status' => 'processing',
      'total_amount' => 1249.9,
      'notes'        => 'Hediye paketi',
      'created_at'   => '2026-10-05',
      'customer'     => 'Ayşe Demir',
  ],
]
```

(e) 2. turda gönderilen messages dizisi

```
[
  {"role": "user", "content": "ORD-1003 siparişinin durumu ne?"},
  {"role": "assistant", "content": [
      {"type": "tool_use", "id": "toolu_01A", "name": "get_order_status",
       "input": {"order_number": "ORD-1003"}}]},
  {"role": "user", "content": [
      {"type": "tool_result", "tool_use_id": "toolu_01A", "is_error": false,
       "content": "{\"status\":\"success\",\"order\":{\"order_number\":\"ORD-1003\",
                   \"order_status\":\"processing\", ... ,\"customer\":\"Ayşe Demir\"}}"}]}
]
```

(f) 2. tur yanıtı → Controller → Tarayıcı

```
// Anthropic: {"stop_reason": "end_turn",
//             "content": [{"type": "text", "text": "ORD-1003 numaralı sipariş ..."}]}

HTTP/1.1 200 OK
{"reply": "ORD-1003 numaralı sipariş Ayşe Demir'e ait ve şu an hazırlanıyor. Tutar: 1.249,90 TL."}
```

3.3 Paralel / çok adımlı tool kullanımı

Kullanıcı tek mesajda iki şey sorarsa (“Ahmet Yılmaz'ın siparişleri ve ORD-1003'ün durumu?”), model aynı yanıtta **iki** `tool_use` bloğu döndürebilir. `AiAgent` bu durumu doğal olarak destekler: `foreach` tüm blokları sırayla çalıştırır ve iki `tool_result`'ı **aynı** `user` mesajında gönderir. Bu, API'nin beklediği formattır: paralel tool çağrılarının sonuçları tek mesajda toplanmalıdır.

Model ilk sonuca bakıp ikinci bir tool'a karar da verebilir (ör. önce müşteri arar, sonra belirli bir siparişi sorgular). Bu durumda her karar ayrı bir tur tüketir. `maxSteps = 5` sınırı bu zincirin uzunluğunu kısıtlar.

3.4 Hata akışları

Durum	Nerede yakalanır	HTTP	Kullanıcının gördüğü
CSRF token yok/geçersiz	web middleware (VerifyCsrfToken)	419	“CSRF token mismatch.” (Laravel mesajı)
`message` boş, string değil, >2000 karakter	`$request->validate()`	422	Doğrulama mesajı (ör. alan zorunlu)
API anahtarı hatalı, model adı yanlış, rate limit, aşırı yük (401/400/429/529)	`->throw()` → `RequestException` → controller	502	“AI servisine ulaşılamadı. Lütfen tekrar dene.”
30 sn timeout, DNS/bağlantı hatası	`ConnectionException` → `Throwable` dalı	500	“Beklenmeyen bir hata oluştu.”
Kod hatası (TypeError, DB bağlantısı vb.)	`Throwable` dalı	500	“Beklenmeyen bir hata oluştu.”
Model bilinmeyen tool adı ister	`ToolsService::call()` whitelist	200	Model hatayı `is_error` ile alır, kullanıcıya açıklar
Müşteri/sipariş bulunamadı	Tool metodu `status: error` döner	200	“… bulunamadı” (modelin ifadesiyle)
5 turda cevap çıkmadı	`AiAgent::run()` sonrası fallback	200	“Bu soruyu yanıtlamak için çok fazla adım gerekti…”
Model boş metin döndürdü	`AiAgent::text(...) ?: …`	200	“Bir cevap üretilemedi.”

Tasarım ilkesi: **İş seviyesindeki “olumsuz” sonuçlar (kayıt yok, belirsiz isim) modele veri olarak verilir, 200 ile döner. Yalnızca altyapı arızaları HTTP hata koduna dönüşür.** Böylece model “bulunamadı” bilgisini kullanıcıya bağlamıyla birlikte, doğal bir dille aktarabilir.

3.5 Süre ve maliyet profili

- Her tur **bir** API çağrısıdır. Tipik soru 2 tur sürer: tool isteği + nihai cevap.
- Her çağrıda `system` prompt ve iki tool şeması **yeniden** gönderilir; geçmiş de her turda büyür. Token maliyeti turlarla birlikte artar.
- En kötü durumda 5 tur × 30 sn timeout = **150 sn**. Bu süre PHP-FPM'in `max_execution_time` değerini ve web sunucusu/proxy zaman aşımlarını aşabilir (bkz. 6.6 Bilinen sınırlamalar).
- `max_tokens: 1024` tek bir yanıtın uzunluğunu sınırlar. Kısa müşteri hizmetleri cevapları için yeterlidir.

4. Klasör Yapısı

4.1 İlgili dizin ağacı

```
exampleai/
|-- app/
|   |-- Http/
|   |   `-- Controllers/
|   |       |-- Controller.php              (boş abstract taban sınıf)
|   |       `-- Ai/
|   |           `-- AgentController.php     <- HTTP giriş noktası
|   |-- Schema/
|   |   `-- ToolSchema.php                  <- Modele sunulan tool sözleşmeleri
|   |-- Services/
|   |   `-- Ai/
|   |       |-- AiAgent.php                 <- Ajan döngüsü / orkestrasyon
|   |       |-- AiClient.php                <- Anthropic HTTP istemcisi
|   |       `-- ToolsService.php            <- Tool uygulamaları (Eloquent)
|   `-- Models/
|       |-- Customer.php                    (hasMany Order)
|       `-- Order.php                       (belongsTo Customer)
|-- config/
|   `-- services.php                        ('anthropic' => key, model)
|-- database/migrations/
|   |-- 2026_10_12_000001_create_customers_table.php
|   `-- 2026_10_12_000002_create_orders_table.php
|-- resources/views/
|   |-- ai/agent.blade.php                  (sayfa: <x-ai.chat :endpoint=...>)
|   `-- components/ai/chat.blade.php        (sohbet UI + fetch)
`-- routes/
    `-- web.php                             (ai.agent, ai.send)
```

4.2 Her klasörün görevi

app/Http/Controllers/Ai/

HTTP katmanı. Dış dünyadan gelen isteği karşılar, doğrular ve iç servislere devreder. `Ai` alt klasörü, AI ile ilgili controller'ları diğer controller'lardan ayırır. Namespace `App\Http\Controllers\Ai` olur (PSR-4: klasör yolu = namespace). İleride `ChatController` veya `AiAdminController` gibi sınıflar da buraya eklenir. Bu, **özellik (feature) bazlı gruplamanın** küçük bir uygulamasıdır.

app/Services/Ai/

Uygulama / domain servis katmanı. Laravel'de varsayılan bir `Services` klasörü yoktur; bu bir topluluk konvansiyonudur. Amaç, controller'ı ince tutup iş mantığını HTTP'den bağımsız, yeniden kullanılabilir sınıflara taşımaktır. Buradaki sınıflar bir Artisan komutundan, bir queue job'undan veya testlerden de çağrılabilir; hiçbiri `Request` nesnesine bağımlı değildir. Composer'ın PSR-4 autoload'u sayesinde ekstra kayıt gerekmez; container sınıfları otomatik bulur.

Klasördeki üç dosya, ajan mimarisinin üç temel rolünü temsil eder: **orkestratör** (AiAgent), **model erişimi** (AiClient) ve **araç uygulamaları** (ToolsService).

app/Schema/

Sözleşme / tanım katmanı. Davranış değil, **veri tanımı** içerir. Tool şemaları hem modele gönderilen bir sözleşme hem de güvenlik whitelist'idir. Bu yüzden servislerden ayrı, kendi klasöründe tutulmuştur. Laravel standardında olmayan bir klasördür.

> **İsim benzerliği uyarısı**
> `App\Schema` namespace'i, Laravel'in `Illuminate\Support\Facades\Schema` facade'ı ile isim olarak benzerdir. Farklı namespace'lerde oldukları için çakışma olmaz. Yine de bir migration veya serviste ikisi birlikte import edilirse `use ... as ...` ile takma ad vermek gerekir.

Akışa dahil diğer klasörler

- `routes/web.php`: AI uç noktalarının URL'lerini ve route adlarını (`ai.agent`, `ai.send`) tanımlar.
- `config/services.php`: Anthropic anahtarı ve model adını `.env`'den okur. Kod `env()` yerine her zaman `config()` kullanır; böylece `php artisan config:cache` ile uyumludur.
- `app/Models/`: Tool'ların sorguladığı Eloquent modelleri.
- `resources/views/components/ai/`: Sohbet arayüzü. Endpoint, controller'ın döndürdüğü `{reply}` sözleşmesine göre yazılmıştır.

5. Dosya Açıklamaları

Bu bölüm her dosyayı “kod okumadan” anlaşılacak şekilde tanıtır. Satır satır inceleme 6. bölümdedir.

5.1 AgentController

`app/Http/Controllers/Ai/AgentController.php`

Başlık	Açıklama
Amaç	AI asistanın HTTP kapısı. Sohbet sayfasını sunar ve mesaj uç noktasını sağlar.
Neden oluşturuldu?	HTTP'ye özgü işleri (doğrulama, durum kodları, JSON, loglama) ajan mantığından ayırmak için. AiAgent HTTP'yi, controller ise LLM'i bilmez.
Hangi problemi çözüyor?	Kullanıcı girdisinin güvenle alınması ve dış servis hatalarının kullanıcıya anlamlı, iç detay sızdırmayan mesajlarla dönmesi.
Sistemdeki görevi	“İnce controller”: doğrula → ajanı çalıştır → sonucu/hatayı HTTP'ye çevir.
Kullananlar	`routes/web.php` (`ai.agent` → `index`, `ai.send` → `post`). Dolaylı olarak `components/ai/chat.blade.php` (fetch ile).
Kullandıkları	`App\Http\Controllers\Controller` (taban sınıf), `AiAgent`, `Request`, `RequestException`, `Log`, `Throwable`, `ai.agent` view'ı.

Önemli metodlar / üyeler

Metod	Açıklama
`index()`	`GET /ai/agent`. `resources/views/ai/agent.blade.php` view'ını döner. Mantık içermez.
`post(Request, AiAgent)`	`POST /ai/agent/send`. Mesajı doğrular, `AiAgent::run()`'ı tek kullanıcı mesajıyla çağırır, API hatalarını 502'ye, diğer tüm hataları 500'e çevirir, başarıda `{reply}` döner.

5.2 AiAgent

`app/Services/Ai/AiAgent.php`

Başlık	Açıklama
Amaç	Sistemin orkestratörü. Modelle tool'lar arasındaki çok adımlı konuşmayı yönetir.
Neden oluşturuldu?	Tool-use protokolü tek bir API çağrısıyla bitmez: istek → tool → sonuç → tekrar istek. Bu döngünün, persona kurallarının ve adım sınırının tek bir yerde toplanması gerekir.
Hangi problemi çözüyor?	Modelin tool isteklerini çalıştırıp sonuçları API'nin beklediği formatta geri beslemek, sonsuz döngüyü engellemek ve modelin veri uydurmasını (hallucination) system prompt ile kısıtlamak.
Sistemdeki görevi	Controller ile alt servisler (AiClient, ToolsService) arasında koordinatör.
Kullananlar	`AgentController` (`run()` ve statik `text()`).
Kullandıkları	`AiClient` (inject), `ToolsService` (inject), `ToolSchema` (statik), Laravel `collect()` helper.

Önemli metodlar / üyeler

Metod	Açıklama
`$system`	Persona ve davranış kuralları (Türkçe, tool dışı bilgi uydurma, durum çevirileri, TL).
`$tools`	`ToolSchema::all()` çıktısı; her API çağrısında modele gönderilir.

Metod	Açıklama
`$maxSteps = 5`	Bir istekteki en fazla API turu sayısı.
`__construct(AiClient, ToolsService)`	Constructor property promotion ile bağımlılıkları alır, tool listesini yükler.
`run(array $messages): array`	Ajan döngüsü. Son Anthropic yanıtını (veya aynı formatta sentetik bir yanıtı) döner.
`static text(array $response): string`	Yanıttaki tüm `text` bloklarını satır sonuyla birleştirir.

5.3 AiClient

app/Services/Ai/AiClient.php

Başlık	Açıklama
Amaç	Anthropic Messages API'ye tek bir HTTP isteği atan ince adaptör (gateway).
Neden oluşturuldu?	URL, header, API versiyonu, model, `max_tokens` ve timeout gibi sağlayıcıya özgü ayrıntıları tek bir yerde toplamak için.
Hangi problemi çözüyor?	API versiyonu yükselince, model değişince veya timeout ayarlanınca tek bir dosyanın değişmesi yeter. Testlerde `Http::fake()` ile gerçek ağa çıkmadan sahte yanıt verilebilir.
Sistemdeki görevi	Altyapı katmanı. İş mantığı içermez; ne gönderildiğini değil, *nasıl* gönderildiğini bilir.
Kullananlar	`AiAgent`.
Kullandıkları	`Illuminate\Support\Facades\Http`, `config('services.anthropic.*')`.

Önemli metodlar / üyeler

Metod	Açıklama
`send(array $messages, string $system = '', array $tools = []): array`	Payload'u kurar, boş `system`/`tools` alanlarını göndermez, isteği atar. HTTP hatasında exception fırlatır; başarıda JSON'u dizi olarak döner.

5.4 ToolsService

app/Services/Ai/ToolsService.php

Başlık	Açıklama
Amaç	Modelin istediği tool'ları gerçekten çalıştıran katman (ajanın “elleri”).
Neden oluşturuldu?	Model veritabanına doğrudan erişemez ve erişmemelidir. Erişim, sınırları net, denetlenebilir PHP metotlarıyla sağlanır.
Hangi problemi çözüyor?	Güvenli dinamik dispatch (whitelist), modelden gelen girdinin normalize edilmesi, verinin modele uygun sade bir formata çevrilmesi ve hataların veri olarak dönmesi.
Sistemdeki görevi	Domain/veri erişim katmanı. LLM'den habersizdir; isim + girdi alır, dizi döner.
Kullananlar	`AiAgent::run()`.
Kullandıkları	`ToolSchema` (whitelist), `App\Models\Customer`, `App\Models\Order`.

Önemli metodlar / üyeler

Metod	Açıklama
`call(string $name, array $input): array`	Tek giriş noktası. Adı whitelist ve `method_exists` ile doğrular, ilgili metodu dinamik olarak çağırır.
`get_customer_orders(array $input): array`	İsme göre (LIKE) en fazla 5 müşteri ve siparişlerini, sipariş sayısı ve iptaller hariç toplam harcamayla döner.
`get_order_status(array $input): array`	Sipariş numarasıyla tek siparişi ve müşteri adını döner. “1001” girdisini “ORD-1001”e tamamlar.
`private formatOrder(Order): array`	Her iki tool'un ortak sipariş formatı.

5.5 ToolSchema

app/Schema/ToolSchema.php

Başlık	Açıklama
Amaç	Modele sunulan tool sözleşmelerini (ad, açıklama, girdi JSON Schema'sı) tanımlar.
Neden oluşturuldu?	Anthropic API her istekte `tools` parametresinde bu tanımları bekler. Model hangi tool'u ne zaman ve hangi parametreyle çağıracağına yalnızca bu açıklamalara bakarak karar verir.
Hangi problemi çözüyor?	Tool listesi için tek doğruluk kaynağı sağlar: aynı liste hem modele gönderilir hem de ToolsService'te whitelist olarak kullanılır. İkisi birbirinden kopamaz.
Sistemdeki görevi	Sözleşme / yapılandırma. Davranış içermez.
Kullananlar	`AiAgent::__construct()`, `ToolsService::call()`.
Kullandıkları	Hiçbir şey (saf PHP dizisi).

Önemli metodlar / üyeler

Metod	Açıklama
`static all(): array`	İki tool tanımını döner: `get_customer_orders` (girdi: `customer_name`) ve `get_order_status` (girdi: `order_number`).

6. Kod Analizleri

Her dosya, gerçek satır numaralarıyla parça parça incelenir. Her parçanın altında kodun **ne yaptığı** ve daha önemlisi **neden böyle yazıldığı** açıklanır.

6.1 AgentController.php

app/Http/Controllers/Ai/AgentController.php · satır 3–10

```
3  namespace App\Http\Controllers\Ai;
4
5  use App\Http\Controllers\Controller;
6  use App\Services\Ai\AiAgent;
7  use Illuminate\Http\Client\RequestException;
8  use Illuminate\Http\Request;
9  use Illuminate\Support\Facades\Log;
10 use Throwable;
```

Namespace klasör yapısıyla birebir aynıdır (PSR-4). Composer bu sayede sınıfı ek kayıt olmadan bulur.

`RequestException` özellikle import edilmiştir, çünkü **API'nin hata döndürmesi** ile **diğer tüm hatalar** ayrı ele alınır. `Throwable` ise `Exception`'dan daha geniştir: PHP'nin `Error` sınıflarını (TypeError, Error vb.) da yakalar. Böylece hiçbir hata kullanıcıya ham stack trace olarak sızmaz.

app/Http/Controllers/Ai/AgentController.php · satır 12–16

```
12 class AgentController extends Controller
13 {
14     public function index(){
15         return view('ai.agent');
16     }
```

`extends Controller`: Laravel 11+ ile taban `Controller` sınıfı boş bir abstract sınıftır. Miras almak, ileride tüm controller'lara ortak bir davranış eklenebilmesi için tutarlılık sağlar.

`index()` yalnızca view döner. Sayfa render'ı ile mesaj işleme ayrı action'lardadır. Böylece `GET` isteği AiAgent'ı oluşturma maliyetine (container çözümlemesi) hiç girmez.

app/Http/Controllers/Ai/AgentController.php · satır 21

```
21     public function post(Request $request, AiAgent $aiAgent){
```

Method injection (Dependency Injection). `AiAgent` parametresi Laravel'in Service Container'ı tarafından otomatik oluşturulur (autowiring). Container reflection ile `AiAgent`'ın constructor'ına bakar, `AiClient` ve `ToolsService`'i oluşturur, onları da enjekte eder.

Neden DI? (1) Controller, AiAgent'ın neye bağımlı olduğunu bilmek zorunda kalmaz. Yarın AiAgent'a bir `Logger` bağımlılığı eklense controller değişmez. (2) Testte `$this->mock(AiAgent::class, ...)` ile ajan sahtesiyle değiştirilebilir; gerçek API'ye gidilmez.

Neden constructor yerine method injection? Bağımlılığa yalnızca `post()` ihtiyaç duyar. Constructor'a koymak `index()` çağrısında da gereksiz yere nesne oluşturur.

app/Http/Controllers/Ai/AgentController.php · satır 22–25

```
22         $request->validate([
23             'message' => 'required|string|max:2000',
24         ]);
25         $message = $request->message;
```

Validasyon neden gerekli? `required`: boş mesajla API'ye para harcayan bir çağrı yapılmasını engeller. `string`: `message[]=x` gibi dizi girdileri reddeder; aksi halde Anthropic'e geçersiz `content` gider. `max:2000`: token maliyetini ve kötüye kullanımı (çok uzun prompt, prompt injection yüzeyi) sınırlar.

İstek `Accept: application/json` taşıdığı için doğrulama hatası redirect yerine otomatik olarak **422 JSON** olur. Chat bileşeni bu JSON'daki `message` alanını hata balonu olarak gösterir.

`$request->message` sihirli property erişimidir (`$request->input('message')` ile eşdeğer). Daha açık bir alternatif: `$message = $request->validate([...])['message'];`. Bu yalnızca doğrulanmış veriyi kullanır.

app/Http/Controllers/Ai/AgentController.php · satır 27–33

```
27          try {
28              $response = $aiAgent->run([
29                  [
30                      'role' => 'user',
31                      'content' => $message,
32                  ]
33              ]);
```

Ajan, Anthropic formatında tek elemanlı bir mesaj dizisiyle başlatılır. `content` düz string verilebilir; API bunu tek bir `text` bloğu olarak yorumlar.

Geçmiş eklenmez, yani her istek bağımsızdır (stateless). Çok turlu bir sohbet için frontend'in tüm geçmişi göndermesi ya da sunucunun oturumda/DB'de saklaması gerekir.

app/Http/Controllers/Ai/AgentController.php · satır 34–37

```
34          } catch (RequestException $e) {
35              Log::error('Anthropic API hatası', ['status' => $e->response->status(), 'body' => $e->respon
 »  se->json()]);
36
37              return response()->json(['message' => 'AI servisine ulaşılamadı. Lütfen tekrar dene.'], 502)
 »  ;
```

`RequestException`, AiClient'taki `->throw()` tarafından 4xx/5xx yanıtlarda fırlatılır. Loga hem **status** hem **body** yazılır. Anthropic hata gövdesi (`{type: 'error', error: {type, message}}`) teşhis için kritiktir: 401 geçersiz anahtar, 400 yanlış model adı, 429 rate limit, 529 aşırı yük.

Kullanıcıya genel bir mesaj döner; API detayları (anahtar hatası vb.) sızdırılmaz. **502 Bad Gateway** anlamsal olarak doğrudur: sunucumuz çalışıyor, ama arkasındaki (upstream) servis başarısız oldu.

app/Http/Controllers/Ai/AgentController.php · satır 38–41

```
38          } catch (Throwable $e) {
39              Log::error('AI agent hatası', ['exception' => $e]);
40
41              return response()->json(['message' => 'Beklenmeyen bir hata oluştu.'], 500);
```

Diğer her şey 500 olur. `['exception' => $e]` bağlamı, Laravel log'unda tam stack trace'in yazılmasını sağlar.

Dikkat: Zaman aşımı ve bağlantı hataları `Illuminate\Http\Client\ConnectionException` fırlatır. Bu sınıf `RequestException` **değildir**, dolayısıyla bu dala düşer ve kullanıcı 502 yerine 500 / "Beklenmeyen bir hata" görür. Ayrı bir `catch (ConnectionException $e)` eklemek davranışı daha tutarlı yapar.

app/Http/Controllers/Ai/AgentController.php · satır 44–46

```
44          return response()->json([
45              'reply' => AiAgent::text($response) ?: 'Bir cevap üretilemedi.',
46          ]);
```

Ham Anthropic yanıtı `AiAgent::text()` ile düz metne çevrilir. `?:` (Elvis) operatörü boş string durumunda yedek mesaja düşer. Bu durum, örneğin `max_tokens` sınırına metin üretilmeden takılan bir yanıtta ortaya çıkar.

Dönüş şekli `{reply}`, chat bileşenindeki `extractReply()` fonksiyonunun ilk kontrol ettiği alandır (`typeof data?.reply === 'string'`). Backend ve frontend bu küçük sözleşmeyle bağlıdır.

6.2 AiAgent.php

app/Services/Ai/AiAgent.php · satır 1–7

```
1  <?php
2
3  namespace App\Services\Ai;
4
5  use App\Schema\ToolSchema;
6
7  class AiAgent {
```

`ToolSchema` doğrudan import edilir; tool listesi enjekte edilmez, statik olarak okunur. Liste sabit ve durumsuz olduğu için bu kabul edilebilir bir sadeleştirmedir.

Sınıf bir **interface** uygulamaz. Tek bir ajan türü olduğu için soyutlama şu an gereksizdir (YAGNI). Ne zaman gerekeceği 6.6'da tartışılmıştır.

app/Services/Ai/AiAgent.php · satır 9–13

```
9       private string $system = 'Sen bir e-ticaret mağazasının müşteri hizmetleri asistanısın. ' .
10          'Müşteri ve sipariş bilgilerini yalnızca sana verilen tool\'larla öğren; tool sonucunda olmayan
» bir bilgiyi asla uydurma. ' .
11          'Bir kayıt bulunamazsa bunu açıkça söyle. Birden fazla müşteri eşleşirse hangisini kastettiğini
» sor ya da hepsini kısaca listele. ' .
12          'Sipariş durumlarını Türkçe ifade et: pending = beklemede, processing = hazırlanıyor, completed
» = tamamlandı, cancelled = iptal edildi. ' .
13          'Tutarları TL olarak yaz. Her zaman Türkçe, kısa ve net cevap ver.';
```

System prompt, modelin kimliğini ve kurallarını belirler. Her cümlenin bir amacı vardır:

- *"tool'larla öğren; uydurma"* → Hallucination'a karşı ana savunma. Model veritabanındaki bilgiyi ancak tool sonucundan bilebilir.
- *"Bir kayıt bulunamazsa açıkça söyle"* → Tool'ların `status: error` dönüşlerinin nasıl ele alınacağını tarif eder.
- *"Birden fazla müşteri eşleşirse …"* → `get_customer_orders` LIKE ile 5'e kadar müşteri döndürebildiği için gereken belirsizlik giderme kuralı. Kod ile prompt burada birbirini tamamlar.
- *Durum çevirileri* → Veritabanı enum'u İngilizcedir (`pending` …). Çeviri kodda değil promptta yapılır; böylece tool çıktısı ham ve dilden bağımsız kalır.
- *"TL, Türkçe, kısa ve net"* → Ton ve format; `max_tokens: 1024` ile de uyumludur.

Prompt'un config yerine kodda tutulması, prompt değişikliklerinin kodla birlikte versiyonlanmasını sağlar. Prompt'u değiştirmek bir kod değişikliğidir; review ve test sürecinden geçer.

app/Services/Ai/AiAgent.php · satır 15–25

```
15      private array $tools;
16
17      private int $maxSteps = 5;
18
19      public function __construct(
20          private AiClient $aiClient,
21          private ToolsService $toolsService,
22      )
23      {
24          $this->tools = ToolSchema::all();
25      }
```

Constructor property promotion (PHP 8). `private AiClient $aiClient` hem property'yi tanımlar hem atamayı yapar. Daha az tekrar ve daha okunur bir kod elde edilir.

Constructor injection. Ajanın bu iki servis olmadan çalışamayacağı imzadan bellidir (açık bağımlılık). İkisi de somut sınıftır ve constructor'larında parametre yoktur. Bu yüzden container herhangi bir `bind()` kaydı olmadan onları otomatik oluşturur.

`$maxSteps = 5`: Tipik soru 1–2 tool turu ister; 5, birden fazla tool zinciri için pay bırakırken maliyeti, gecikmeyi ve olası bir modelin aynı tool'u tekrar tekrar çağırma döngüsünü sınırlar.

`$this->tools = ToolSchema::all()` nesne başına bir kez çalışır. Her turda yeniden oluşturulmaz.

app/Services/Ai/AiAgent.php · satır 30–37

```
30      public function run(array $messages): array
31      {
32          for ($step = 0; $step < $this->maxSteps; $step++) {
33              $response = $this->aiClient->send($messages, $this->system, $this->tools);
34
35              if (($response['stop_reason'] ?? null) !== 'tool_use') {
36                  return $response;
37              }
```

`run()` **string değil dizi döner**: ham Anthropic yanıtı. Böylece çağıran taraf `stop_reason`, `usage` (token sayıları) gibi alanlara erişebilir. Metne çevirme işi ayrı bir sorumluluk olarak `text()`'e bırakılmıştır.

Döngü sayaçlıdır (`for`), `while(true)` değildir; sınır yapısal olarak garanti altındadır.

`($response['stop_reason'] ?? null) !== 'tool_use'`: `end_turn`, `max_tokens`, `stop_sequence`, `refusal` gibi **tool dışındaki tüm durumlar** "bitti" sayılır. `?? null` beklenmedik bir yanıtta "undefined index" uyarısını engeller (savunmacı programlama).

app/Services/Ai/AiAgent.php · satır 39–43

```
39              // Modelin yanıtı (tool_use blokları dahil) olduğu gibi geçmişe eklenir.
40              $messages[] = [
41                  'role' => 'assistant',
42                  'content' => $response['content'],
43              ];
```

Modelin yanıtı **değiştirilmeden** `assistant` mesajı olarak eklenir. Bu API'nin zorunlu bir kuralıdır: `tool_result` blokları bir önceki assistant mesajındaki `tool_use` id'lerine referans verir. Blokları filtrelemek veya yeniden yazmak protokolü bozar ve API 400 döndürür.

app/Services/Ai/AiAgent.php · satır 45–61

```
45              // Her tool_use bloğu çalıştırılır, sonuçlar tek bir user mesajıyla geri gönderilir.
46              $toolResults = [];
47
48              foreach ($response['content'] as $block) {
49                  if (($block['type'] ?? null) !== 'tool_use') {
50                      continue;
51                  }
52
53                  $result = $this->toolsService->call($block['name'], (array) ($block['input'] ?? []));
54
55                  $toolResults[] = [
56                      'type' => 'tool_result',
57                      'tool_use_id' => $block['id'],
58                      'content' => json_encode($result, JSON_UNESCAPED_UNICODE),
59                      'is_error' => ($result['status'] ?? null) === 'error',
60                  ];
61              }
```

`content` dizisinde metin blokları da olabilir (model tool çağırmadan önce "Kontrol ediyorum…" diyebilir). `continue` yalnızca `tool_use` bloklarını işler.

`(array) ($block['input'] ?? [])`: Input yoksa veya boş bir JSON nesnesi (`{}`) gelirse bile ToolsService'in `array` tip ipucu (type hint) TypeError fırlatmaz.

`tool_use_id => $block['id']`: Her sonucu doğru isteğe bağlar. Paralel çağrılarda eşleşme buna dayanır.

`json_encode(..., JSON_UNESCAPED_UNICODE)`: Tool sonucu API'ye string olarak verilir. Bu bayrak sayesinde "Ayşe" `Ay\u015fe` olmaz; hem okunur kalır hem de daha az token harcar.

`is_error`: ToolsService'in `status: error` sözleşmesi API'nin `is_error` alanına çevrilir. Model böylece sonucun başarısız olduğunu kesin olarak bilir.

app/Services/Ai/AiAgent.php · satır 63–67

```
63              $messages[] = [
64                  'role' => 'user',
65                  'content' => $toolResults,
66              ];
67          }
```

Tüm `tool_result` blokları **tek bir** `user` mesajında gönderilir. API, paralel tool çağrılarının sonuçlarını tek mesajda bekler; ayrı ayrı mesajlar hem kurala aykırıdır hem de modelin paralel tool kullanma eğilimini bozar. Ardından döngü başa döner ve büyümüş `$messages` ile yeni bir istek atılır.

app/Services/Ai/AiAgent.php · satır 69–74

```
69          return [
70              'stop_reason' => 'max_steps',
71              'content' => [
72                  ['type' => 'text', 'text' => 'Bu soruyu yanıtlamak için çok fazla adım gerekti. Lütfen s
 » oruyu daha net sorar mısın?'],
73              ],
74          ];
```

Adım sınırı aşılırsa exception fırlatılmaz. Bunun yerine **Anthropic yanıtıyla aynı şekle sahip** sentetik bir dizi döner (`stop_reason` + `content[text]`). Bu bir tür *Null Object* yaklaşımıdır: controller'ın özel bir durumu ele alması gerekmez, `text()` aynen çalışır ve kullanıcı yönlendirici bir mesaj görür.

app/Services/Ai/AiAgent.php · satır 80–86

```
80      public static function text(array $response): string
81      {
82          return collect($response['content'] ?? [])
83              ->where('type', 'text')
84              ->pluck('text')
85              ->implode("\n");
86      }
```

Laravel Collection zinciri: `where('type', 'text')` metin bloklarını filtreler, `pluck('text')` metinleri çeker, `implode("\n")` birleştirir. Model bir yanıtta birden fazla metin bloğu üretebilir (ör. tool öncesi ve sonrası). Bu metot hepsini birleştirir.

Neden statik? Saf bir fonksiyondur: nesne durumuna ihtiyaç duymaz, aynı girdiye hep aynı çıktıyı verir. Ajan örneğine sahip olmayan bir yerden (ör. bir test) de çağrılabilir.

6.3 AiClient.php

app/Services/Ai/AiClient.php · satır 1–8

```
1  <?php
2
3  namespace App\Services\Ai;
4
5  use Illuminate\Support\Facades\Http;
6
7  class AiClient {
8      public function send(array $messages , string $system = "" ,  array $tools = []):array{
```

`Http` facade, Laravel'in Guzzle tabanlı HTTP istemcisidir. Facade kullanımı, testlerde `Http::fake()` ile tüm dış çağrıların sahteleriyle değiştirilebilmesini sağlar.

`send()` imzasında `system` ve `tools` opsiyoneldir. İstemci böylece tool'suz basit tek seferlik çağrılar (ör. bir özetleme özelliği) için de yeniden kullanılabilir. Ajana özgü hiçbir şey içermez.

app/Services/Ai/AiClient.php · satır 10–14

```
10          $payload  = [
11              'model' => config('services.anthropic.model'),
12              'messages' => $messages, // dizi olacak ve içeriği role ve content olacak şekilde
13              'max_tokens' => 1024,
14          ];
```

`model` **config'ten** okunur, kodda sabit yazılı değildir. Model yükseltmesi yalnızca `.env` değişikliğiyle yapılır. Kod `env()`'i doğrudan çağırmaz, `config()` kullanır; bu, config cache ile uyumluluk için Laravel'in önerdiği yoldur.

`max_tokens` Messages API'de **zorunlu** bir parametredir. 1024, kısa müşteri hizmetleri cevapları için yeterli bir üst sınırdır ve maliyeti de kontrol eder.

app/Services/Ai/AiClient.php · satır 16–21

```
16          if ($system !== '') {
17              $payload['system'] = $system;
18          }
19          if (!empty($tools)) {
20              $payload['tools'] = $tools;
21          }
```

Boş `system` ve boş `tools` alanları payload'a hiç eklenmez. API boş değerlere doğrulama hatası verebileceği için alanı hiç göndermemek hem daha temiz hem daha güvenlidir.

app/Services/Ai/AiClient.php · satır 23–29

```
23          return Http::withHeaders([
24              'x-api-key' => config('services.anthropic.key'),
25              "anthropic-version" => "2023-06-01"
26          ])
27          ->timeout(30)
28          ->post('https://api.anthropic.com/v1/messages', $payload
29          )->throw()->json();
```

`x-api-key`: Anthropic kimlik doğrulama header'ı. Değer config'ten gelir, koda veya loglara yazılmaz.

`anthropic-version: 2023-06-01`: Zorunlu header. API davranışını belirli bir sözleşme sürümüne sabitler. Böylece Anthropic tarafındaki değişiklikler mevcut kodu habersizce bozmaz.

`timeout(30)`: Tek bir çağrının en fazla 30 sn sürmesine izin verir (Laravel'in varsayılanı da 30 sn'dir; açıkça yazmak niyeti belgeler).

`post($url, $payload)`: Dizi payload otomatik olarak JSON'a çevrilir ve `Content-Type: application/json` eklenir.

`->throw()`: **Kritik satır.** Laravel HTTP istemcisi 4xx/5xx yanıtlarda varsayılan olarak exception fırlatmaz. `throw()` olmasaydı hata gövdesi normal yanıt gibi `json()` ile dönerdi. `stop_reason` olmadığı için AiAgent bunu "bitti" sayar, kullanıcı da sessizce "Bir cevap üretilemedi." görürdü. `throw()` hatayı görünür kılar ve controller'ın 502 dalına yönlendirir.

`->json()`: Yanıt gövdesini ilişkisel PHP dizisine çevirir. Sistemin geri kalanı bu diziyle çalışır.

> **Bilinçli olarak eklenmemiş özellikler**
> Retry yok: 429/529 gibi geçici hatalar için `->retry(2, 500, fn ($e) => ...)` eklenebilir. Interface yok: sağlayıcı değiştirmek veya testte sahte istemci kullanmak için bir `LlmClient` interface'i ve `AppServiceProvider` içinde binding eklenebilir. Şu an `Http::fake()` test ihtiyacını karşılar.

6.4 ToolsService.php

app/Services/Ai/ToolsService.php · satır 1–10

```
1  <?php
2
3  namespace App\Services\Ai;
4
5  use App\Models\Customer;
6  use App\Models\Order;
7  use App\Schema\ToolSchema;
8
9  class ToolsService
10 {
```

Servis yalnızca modelleri ve `ToolSchema`'yı import eder; LLM, HTTP veya AiClient ile hiçbir bağı yoktur. Bu nedenle bir Artisan komutundan veya testten doğrudan çağrılabilir.

app/Services/Ai/ToolsService.php · satır 14–23

```
14      public function call(string $name, array $input): array
15      {
16          $allowed = array_column(ToolSchema::all(), 'name');
17
18          if (! in_array($name, $allowed, true) || ! method_exists($this, $name)) {
19              return ['status' => 'error', 'message' => "Bilinmeyen tool: {$name}"];
20          }
21
22          return $this->{$name}($input);
23      }
```

Güvenli dinamik dispatch. `$this->{$name}($input)` ifadesi, string olarak gelen ismi metod çağrısına çevirir. Böylece yeni bir tool eklemek için `switch`/`match` güncellemek gerekmez. Ancak isim **modelden**, yani dolaylı

olarak kullanıcıdan gelir. Prompt injection ile model `call`, `formatOrder` veya `__construct` gibi bir isim üretmeye zorlanabilir.

İki katmanlı kontrol bunu engeller: (1) `in_array($name, $allowed, true)` yalnızca ToolSchema'daki adlara izin verir. `true` parametresi katı (strict) karşılaştırma yaparak tip hokkabazlığını önler. (2) `method_exists` ise şemaya eklenip ToolsService'e henüz yazılmamış bir tool için PHP fatal error yerine kontrollü bir hata döndürür.

Hata durumunda exception değil, sözleşmeye uygun bir dizi döner. AiAgent bunu `is_error: true` ile modele iletir.

> **Konvansiyon: snake_case metodlar**
> Metod adları `get_customer_orders` gibi snake_case'tir. Bu, PSR-1'in camelCase önerisine aykırıdır ama bilinçlidir: **tool adı = metod adı** konvansiyonu dinamik dispatch'in temelidir. Bir tool'un adını değiştirirken iki yeri (ToolSchema `name` ve metod adı) birlikte değiştirmek gerekir.

app/Services/Ai/ToolsService.php · satır 28–34

```
28      public function get_customer_orders(array $input): array
29      {
30          $customerName = trim((string) ($input['customer_name'] ?? ''));
31
32          if ($customerName === '') {
33              return ['status' => 'error', 'message' => 'Müşteri adı boş olamaz.'];
34          }
```

Girdi normalizasyonu. `input_schema` modelden `customer_name` beklese de model çıktısı %100 garanti değildir: alan eksik, `null` veya sayı olabilir. `?? ''` eksik alanı, `(string)` tip dönüşümünü, `trim()` boşlukları ele alır. Boş isim, bütün müşterileri döndürecek bir `LIKE '%%'` sorgusunu engeller.

app/Services/Ai/ToolsService.php · satır 36–43

```
36          $customers = Customer::where('name', 'like', "%{$customerName}%")
37              ->with(['orders' => fn ($query) => $query->latest()])
38              ->limit(5)
39              ->get();
40
41          if ($customers->isEmpty()) {
42              return ['status' => 'error', 'message' => 'Müşteri bulunamadı.'];
43          }
```

`where('name', 'like', "%{$customerName}%")`: Kısmi eşleşme. “Mehmet” tüm Mehmet'leri, “Yılmaz” tüm Yılmaz'ları bulur. Eloquent parametre binding kullanır; **SQL injection'a karşı güvenlidir**. Ancak girdideki `%` ve `_` karakterleri escape edilmez ve joker karakter gibi davranır (küçük bir doğruluk sorunu, güvenlik sorunu değil).

`with(['orders' => fn ($q) => $q->latest()])`: **Eager loading** ile sınırlandırılmış ilişki. Siparişler en yeniden eskiye sıralanır. 5 müşteri için 1 + 1 = 2 sorgu çalışır; eager loading olmasaydı N+1 problemiyle 6 sorgu çalışırdı.

`limit(5)`: Hem tool açıklamasındaki “en fazla 5” sözünü tutar hem de modele gönderilen veri miktarını (token) sınırlar.

app/Services/Ai/ToolsService.php · satır 45–56

```
45          return [
46              'status' => 'success',
47              'match_count' => $customers->count(),
48              'customers' => $customers->map(fn (Customer $customer) => [
49                  'name' => $customer->name,
50                  'email' => $customer->email,
51                  'city' => $customer->city,
52                  'order_count' => $customer->orders->count(),
53                  'total_spent' => (float) $customer->orders->where('status', '!=', 'cancelled')->sum('tot
 » al_amount'),
54                  'orders' => $customer->orders->map(fn (Order $order) => $this->formatOrder($order))->all
 » (),
55              ])->all(),
56          ];
```

`match_count`: Modelin belirsizlik kuralını (“birden fazla müşteri eşleşirse sor”) uygulamasını kolaylaştırır.

`total_spent`: **İş kuralı** olarak iptal edilen siparişler hariç tutulur. Hesaplama yüklenmiş koleksiyon üzerinde yapılır, ek sorgu gerekmez.

`(float)` dönüşümü: `Order` modelinde `total_amount decimal:2` cast'ine sahiptir ve PHP'de **string** döner (`"1249.90"`). JSON'da sayı olarak görünmesi için float'a çevrilir.

Telefon (`phone`) bilinçli olarak modele gönderilmez (veri minimizasyonu). E-posta ve şehir ise gönderilir. Kişisel verinin üçüncü taraf bir LLM'e iletildiği unutulmamalıdır (bkz. 6.6).

app/Services/Ai/ToolsService.php · satır 62–81

```
62      public function get_order_status(array $input): array
63      {
64          $orderNumber = strtoupper(trim((string) ($input['order_number'] ?? '')));
65
66          // "1001" gibi sadece rakam gelirse "ORD-1001" olarak ara.
67          if (ctype_digit($orderNumber)) {
68              $orderNumber = "ORD-{$orderNumber}";
69          }
70
71          $order = Order::with('customer')->where('order_number', $orderNumber)->first();
72
73          if (! $order) {
74              return ['status' => 'error', 'message' => "{$orderNumber} numaralı sipariş bulunamadı."];
75          }
76
77          return [
78              'status' => 'success',
79              'order' => $this->formatOrder($order) + ['customer' => $order->customer->name],
80          ];
81      }
```

`strtoupper(trim(...))`: “ord-1003 ” → “ORD-1003”. `ctype_digit`: Kullanıcı sadece “1003” yazarsa önek eklenir. Tool şeması `ORD-` formatını istese de servis **toleranslıdır**: modelin veya kullanıcının küçük format hataları başarısız sorguya yol açmaz.

`Order::with('customer')`: Müşteri adı için ikinci sorgu eager loading ile yapılır. `order_number` migration'da `unique` olduğu için indekslidir; arama hızlıdır.

Hata mesajı aranan numarayı içerir. Model “ORD-1003 numaralı sipariş bulunamadı” bilgisini doğrudan kullanıcıya aktarabilir.

`$this->formatOrder($order) + ['customer' => ...]`: PHP **dizi birleşimi (`+`)** operatörü. Ortak formatı tekrar yazmadan tek bir alan ekler.

app/Services/Ai/ToolsService.php · satır 83–92

```
83      private function formatOrder(Order $order): array
84      {
85          return [
86              'order_number' => $order->order_number,
87              'order_status' => $order->status,
88              'total_amount' => (float) $order->total_amount,
89              'notes' => $order->notes,
90              'created_at' => $order->created_at?->toDateString(),
91          ];
92      }
```

Neden private ortak formatlayıcı? İki tool aynı sipariş şeklini döndürür. Model tutarlı alan adlarıyla çalışır, format tek bir yerden değişir. `private` olması onu dış dünyadan gizler (zaten whitelist'te de yoktur).

`'order_status'` anahtarı bilinçlidir: Tool'un üst seviye `status` alanı (`success`/`error`) ile **isim çakışmasını** önler. Model “işlem durumu” ile “sipariş durumu”nu karıştırmaz.

`created_at?->toDateString()`: Nullsafe operatör, `created_at` boşsa hata vermez. Saat bilgisi atılır; daha az token harcanır ve müşteri hizmetleri için gün yeterlidir.

6.5 ToolSchema.php

app/Schema/ToolSchema.php · satır 1–10

```php
<?php
namespace App\Schema;

class ToolSchema
{
    /**
     * Anthropic Messages API tool şemaları.
     */
    public static function all(): array
    {
```

Sınıfın tek işi bir dizi döndürmektir. Durum (state) tutmadığı için statik metod uygundur. Neden `const` değil de metod? Metod, ileride tanımların dinamik üretilmesine (ör. çeviri, config'e göre tool açma/kapama) çağıranları değiştirmeden izin verir.

app/Schema/ToolSchema.php · satır 12–28

```php
            [
                'name' => 'get_customer_orders',
                'description' => 'Veritabanında kayıtlı bir müşterinin sipariş geçmişini listeler: sipar
» iş numarası, durum, tutar, not ve tarih. ' .
                                 'Kullanıcı bir müşterinin siparişlerini, sipariş durumlarını veya topla
» m harcamasını sorduğunda bu aracı kullanın. ' .
                                 'Müşteri adı veya soyadıyla arama yapar ve birden fazla müşteri eşleşeb
» ilir (en fazla 5). ' .
                                 'Sipariş numarası ile arama yapmak için bu araç yerine `get_order_statu
» s` kullanın.',
                'input_schema' => [
                    'type' => 'object',
                    'properties' => [
                        'customer_name' => [
                            'type' => 'string',
                            'description' => 'Sorgulanacak müşterinin adı, soyadı veya ikisi birden (Örn
» : Ahmet Yılmaz, Mehmet)'
                        ]
                    ],
                    'required' => ['customer_name']
                ],
            ],
```

`name`: ToolsService'teki metod adıyla **birebir aynı olmak zorundadır** (dinamik dispatch sözleşmesi).

`description`: Model tool seçimini neredeyse tamamen bu metne bakarak yapar. Açıklama dört şeyi söyler: ne döndürür, **ne zaman** kullanılmalı, nasıl arar (ad/soyad, en fazla 5) ve **ne zaman kullanılmamalı** (“sipariş numarası için `get_order_status` kullanın”). Bu sonuncusu, iki tool arasındaki yanlış seçimleri ciddi ölçüde azaltır. Anthropic de ayrıntılı açıklamaları önerir.

`input_schema`: Standart **JSON Schema**. `type: object`, `properties` içindeki her alanın açıklaması ve örnekleri, modelin doğru formatta argüman üretmesine yardım eder. `required` alanı modelin parametreyi atlamamasını sağlar. Yine de ToolsService savunmacı davranır (bkz. 6.4).

app/Schema/ToolSchema.php · satır 29–44

```
            [
                'name' => 'get_order_status',
                'description' => 'Sipariş numarası vererek tek bir siparişin durumunu (pending, processi
» ng, completed, cancelled), tutarını, notunu, tarihini ve müşterisini getirir. ' .
                                 'Kullanıcı elindeki belirli bir sipariş numarasını sorduğunda bu aracı
» kullanın. ' .
                                 'Müşterinin tüm siparişlerini görmek için bu aracı değil `get_customer_
» orders` kullanın.',
                'input_schema' => [
                    'type' => 'object',
                    'properties' => [
                        'order_number' => [
                            'type' => 'string',
                            'description' => 'Sipariş numarası, "ORD-" önekiyle (Örn: ORD-1001)'
                        ]
                    ],
                    'required' => ['order_number']
                ],
            ],
```

İkinci tool aynı deseni izler. `order_number` açıklamasındaki **örnek format** (`ORD-1001`) modeli doğru biçime yönlendirir. Açıklamadaki durum listesi (`pending, processing, …`) modelin çıktıyı yorumlamasına yardımcı eder; Türkçe karşılıklar ise system prompt'tadır.

6.6 Çapraz konular ve tasarım değerlendirmesi

Dependency Injection: nerede, neden?

Yer	Tür	Gerekçe
`AgentController::post(…, AiAgent $aiAgent)`	Method injection	Bağımlılık sadece bu action'da gerekir; testte mock'lanabilir.
`AiAgent::__construct(AiClient, ToolsService)`	Constructor injection	Ajanın her işleminde gerekli; zorunlu bağımlılıklar imzada açıkça görünür.
`ToolSchema::all()`	Statik çağrı (DI değil)	Durumsuz, sabit veri; enjekte etmenin getirisi düşük.
`Http`, `Log`, `config()`	Facade / helper	Laravel'in test edilebilir facade'ları (`Http::fake`, `Log::spy`).

Hiçbir `bind()` / `singleton()` kaydı yoktur. Tüm sınıflar somut (concrete) ve parametresiz constructor zincirine sahip olduğu için Laravel bunları **sıfır yapılandırmayla** otomatik çözer (autowiring).

Interface kullanımı

Bu modülde interface **yoktur**. Tek bir LLM sağlayıcısı ve tek bir tool seti varken soyutlama eklemek fayda sağlamadan karmaşıklık ekler (YAGNI). Aşağıdaki durumlarda interface eklemek mantıklı hale gelir:

- Birden fazla model sağlayıcısı (ör. Anthropic + başka bir sağlayıcı) desteklenecekse veya ortama göre değişecekse,
- AiAgent'ı testte, HTTP katmanını sahtelemek yerine tamamen sahte bir istemciyle test etmek istenirse.

```
// Örnek (projede YOK, öneridir)
interface LlmClient {
    public function send(array $messages, string $system = '', array $tools = []): array;
}
class AiClient implements LlmClient { /* ... */ }

// AppServiceProvider::register()
$this->app->bind(LlmClient::class, AiClient::class);

// AiAgent constructor'ında AiClient yerine LlmClient tip ipucu kullanılır.
```

> **Önemli**
> Mesaj formatı (`tool_use`, `tool_result`, `stop_reason`) Anthropic'e özgüdür ve AiAgent bu formata doğrudan bağımlıdır. Gerçek bir sağlayıcı bağımsızlığı yalnızca bir interface ile değil, ortak bir mesaj modeline dönüşüm katmanıyla sağlanır.

Trait kullanımı

Bu modülde trait **yoktur** ve bu doğru bir tercihtir. Trait'ler, birbirinden bağımsız birden fazla sınıfın aynı davranışı paylaşması gerektiğinde kullanılır. Burada her sınıfın davranışı kendine özgüdür. Ortak kullanılan tek parça (`formatOrder`) yalnızca ToolsService içinde gerektiği için private metot olarak kalmıştır. Trait eklemek bağımlılıkları gizler ve "bu metod nereden geliyor?" sorusunu zorlaştırır. İleride çok sayıda tool sınıfı olursa ortak yanıt formatı (`success()`/`error()` yardımcıları) için bir trait düşünülebilir.

Güvenlik değerlendirmesi

Konu	Mevcut durum
CSRF	Var. Route `web` grubunda, bileşen `X-CSRF-TOKEN` gönderiyor.
Girdi doğrulama	Var. `required|string|max:2000`.
Tool whitelist	Var. Model yalnızca şemadaki iki tool'u çalıştırabilir.
SQL injection	Korunuyor. Eloquent parametre binding kullanıyor.
Hata detayı sızıntısı	Korunuyor. Kullanıcıya genel mesaj, detay log'a yazılıyor.
API anahtarı	`.env` → config. Kodda ve log'da yok.
Kimlik doğrulama / yetki	**Yok.** `/ai/agent` ve `/ai/agent/send` herkese açık. Siteye erişen herkes müşteri ad/e-posta/sipariş verisini sorgulayabilir ve API kredisi harcayabilir. `auth` middleware'i önerilir.
Rate limiting	**Yok.** `throttle:…` middleware'i ile kullanıcı/IP başına sınır önerilir (maliyet koruması).
Kişisel veri (KVKK)	Müşteri adı, e-posta, şehir ve sipariş verisi üçüncü taraf LLM'e gönderiliyor. Hukuki değerlendirme ve gerekirse veri minimizasyonu yapılmalı.

Bilinen sınırlamalar

- **Sohbet geçmişi yok:** Takip soruları bağlamı kaybeder.
- **Uzun istek süresi:** En kötü durumda ~150 sn (5 × 30 sn). PHP/web sunucusu zaman aşımları önce tetiklenebilir. Streaming yok; kullanıcı yanıt tamamlanana kadar "yazıyor…" görür.
- **ConnectionException 500'e düşüyor:** Timeout, "AI servisine ulaşılamadı" (502) yerine "Beklenmeyen hata" (500) gösterir.
- `max_tokens` **kesilmesi:** `stop_reason: max_tokens` durumunda kısmi metin, uyarı yapılmadan nihai cevap olarak gösterilir.
- `.env.example`**'da Anthropic değişkenleri yok:** Yeni bir kurulumda `ANTHROPIC_API_KEY` / `ANTHROPIC_MODEL` eksik kalırsa API 401/400 döner (kullanıcı 502 görür).
- **LIKE joker karakterleri** (`%`, `_`) escape edilmiyor.
- **Test yok:** `tests/` altında yalnızca varsayılan örnek testler var.

Önerilen test iskeleti

```php
// tests/Feature/AiAgentTest.php (öneri)
Http::fake([
    'api.anthropic.com/*' => Http::sequence()
        ->push(['stop_reason' => 'tool_use', 'content' => [[
            'type' => 'tool_use', 'id' => 'toolu_1', 'name' => 'get_order_status',
            'input' => ['order_number' => 'ORD-1001'],
        ]]])
        ->push(['stop_reason' => 'end_turn', 'content' => [
            ['type' => 'text', 'text' => 'Siparişiniz hazırlanıyor.'],
        ]]),
]);

$this->postJson(route('ai.send'), ['message' => 'ORD-1001 ne durumda?'])
     ->assertOk()
     ->assertJson(['reply' => 'Siparişiniz hazırlanıyor.']);

Http::assertSentCount(2);   // tool turu + nihai cevap
```

7. Dosyaların Tam Kaynak Kodları

Aşağıdaki kodlar proje dosyalarından **değiştirilmeden** alınmıştır. 104 karakterden uzun satırlar sayfaya sığması için bölünmüş ve devam satırları » ile işaretlenmiştir.

7.1 AgentController.php

app/Http/Controllers/Ai/AgentController.php 48 satır · eksiksiz

```
<?php

namespace App\Http\Controllers\Ai;

use App\Http\Controllers\Controller;
use App\Services\Ai\AiAgent;
use Illuminate\Http\Client\RequestException;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;
use Throwable;

class AgentController extends Controller
{
    public function index(){
        return view('ai.agent');
    }

    //agent post


    public function post(Request $request, AiAgent $aiAgent){
        $request->validate([
            'message' => 'required|string|max:2000',
        ]);
        $message = $request->message;

        try {
            $response = $aiAgent->run([
                [
                    'role' => 'user',
                    'content' => $message,
                ]
            ]);
        } catch (RequestException $e) {
            Log::error('Anthropic API hatası', ['status' => $e->response->status(), 'body' => $e->respon
» se->json()]);

            return response()->json(['message' => 'AI servisine ulaşılamadı. Lütfen tekrar dene.'], 502)
» ;
        } catch (Throwable $e) {
            Log::error('AI agent hatası', ['exception' => $e]);

            return response()->json(['message' => 'Beklenmeyen bir hata oluştu.'], 500);
        }

        return response()->json([
            'reply' => AiAgent::text($response) ?: 'Bir cevap üretilemedi.',
        ]);
    }
}
```

7.2 AiAgent.php

app/Services/Ai/AiAgent.php 87 satır · eksiksiz

```php
<?php

namespace App\Services\Ai;

use App\Schema\ToolSchema;

class AiAgent {

    private string $system = 'Sen bir e-ticaret mağazasının müşteri hizmetleri asistanısın. ' .
        'Müşteri ve sipariş bilgilerini yalnızca sana verilen tool\'larla öğren; tool sonucunda olmayan bir bilgiyi asla uydurma. ' .
        'Bir kayıt bulunamazsa bunu açıkça söyle. Birden fazla müşteri eşleşirse hangisini kastettiğini sor ya da hepsini kısaca listele. ' .
        'Sipariş durumlarını Türkçe ifade et: pending = beklemede, processing = hazırlanıyor, completed = tamamlandı, cancelled = iptal edildi. ' .
        'Tutarları TL olarak yaz. Her zaman Türkçe, kısa ve net cevap ver.';

    private array $tools;

    private int $maxSteps = 5;

    public function __construct(
        private AiClient $aiClient,
        private ToolsService $toolsService,
    )
    {
        $this->tools = ToolSchema::all();
    }

    /**
     * Agent döngüsünü çalıştırır ve son Anthropic yanıtını döndürür.
     */
    public function run(array $messages): array
    {
        for ($step = 0; $step < $this->maxSteps; $step++) {
            $response = $this->aiClient->send($messages, $this->system, $this->tools);

            if (($response['stop_reason'] ?? null) !== 'tool_use') {
                return $response;
            }

            // Modelin yanıtı (tool_use blokları dahil) olduğu gibi geçmişe eklenir.
            $messages[] = [
                'role' => 'assistant',
                'content' => $response['content'],
            ];

            // Her tool_use bloğu çalıştırılır, sonuçlar tek bir user mesajıyla geri gönderilir.
            $toolResults = [];

            foreach ($response['content'] as $block) {
                if (($block['type'] ?? null) !== 'tool_use') {
                    continue;
                }

                $result = $this->toolsService->call($block['name'], (array) ($block['input'] ?? []));

                $toolResults[] = [
                    'type' => 'tool_result',
                    'tool_use_id' => $block['id'],
                    'content' => json_encode($result, JSON_UNESCAPED_UNICODE),
                    'is_error' => ($result['status'] ?? null) === 'error',
                ];
            }

            $messages[] = [
                'role' => 'user',
                'content' => $toolResults,
            ];
        }
```

```
68
69          return [
70              'stop_reason' => 'max_steps',
71              'content' => [
72                  ['type' => 'text', 'text' => 'Bu soruyu yanıtlamak için çok fazla adım gerekti. Lütfen s
» oruyu daha net sorar mısın?'],
73              ],
74          ];
75      }
76
77      /**
78       * Anthropic yanıtındaki metin bloklarını birleştirir.
79       */
80      public static function text(array $response): string
81      {
82          return collect($response['content'] ?? [])
83              ->where('type', 'text')
84              ->pluck('text')
85              ->implode("\n");
86      }
87  }
```

7.3 AiClient.php

app/Services/Ai/AiClient.php 31 satır · eksiksiz

```
1   <?php
2
3   namespace App\Services\Ai;
4
5   use Illuminate\Support\Facades\Http;
6
7   class AiClient {
8       public function send(array $messages , string $system = "" ,  array $tools = []):array{
9
10          $payload  = [
11              'model' => config('services.anthropic.model'),
12              'messages' => $messages, // dizi olacak ve içeriği role ve content olacak şekilde
13              'max_tokens' => 1024,
14          ];
15
16          if ($system !== '') {
17              $payload['system'] = $system;
18          }
19          if (!empty($tools)) {
20              $payload['tools'] = $tools;
21          }
22
23          return Http::withHeaders([
24              'x-api-key' => config('services.anthropic.key'),
25              "anthropic-version" => "2023-06-01"
26          ])
27          ->timeout(30)
28          ->post('https://api.anthropic.com/v1/messages', $payload
29          )->throw()->json();
30      }
31  }
```

7.4 ToolsService.php

app/Services/Ai/ToolsService.php 93 satır · eksiksiz

```php
<?php

namespace App\Services\Ai;

use App\Models\Customer;
use App\Models\Order;
use App\Schema\ToolSchema;

class ToolsService
{
    /**
     * Modelin istediği tool'u çalıştırır. Sadece ToolSchema'da tanımlı tool'lar çağrılabilir.
     */
    public function call(string $name, array $input): array
    {
        $allowed = array_column(ToolSchema::all(), 'name');

        if (! in_array($name, $allowed, true) || ! method_exists($this, $name)) {
            return ['status' => 'error', 'message' => "Bilinmeyen tool: {$name}"];
        }

        return $this->{$name}($input);
    }

    /**
     * Tool adı: get_customer_orders
     */
    public function get_customer_orders(array $input): array
    {
        $customerName = trim((string) ($input['customer_name'] ?? ''));

        if ($customerName === '') {
            return ['status' => 'error', 'message' => 'Müşteri adı boş olamaz.'];
        }

        $customers = Customer::where('name', 'like', "%{$customerName}%")
            ->with(['orders' => fn ($query) => $query->latest()])
            ->limit(5)
            ->get();

        if ($customers->isEmpty()) {
            return ['status' => 'error', 'message' => 'Müşteri bulunamadı.'];
        }

        return [
            'status' => 'success',
            'match_count' => $customers->count(),
            'customers' => $customers->map(fn (Customer $customer) => [
                'name' => $customer->name,
                'email' => $customer->email,
                'city' => $customer->city,
                'order_count' => $customer->orders->count(),
                'total_spent' => (float) $customer->orders->where('status', '!=', 'cancelled')->sum('tot
al_amount'),
                'orders' => $customer->orders->map(fn (Order $order) => $this->formatOrder($order))->all
(),
            ])->all(),
        ];
    }

    /**
     * Tool adı: get_order_status
     */
    public function get_order_status(array $input): array
    {
        $orderNumber = strtoupper(trim((string) ($input['order_number'] ?? '')));

        // "1001" gibi sadece rakam gelirse "ORD-1001" olarak ara.
        if (ctype_digit($orderNumber)) {
            $orderNumber = "ORD-{$orderNumber}";
```

```
        }

        $order = Order::with('customer')->where('order_number', $orderNumber)->first();

        if (! $order) {
            return ['status' => 'error', 'message' => "{$orderNumber} numaralı sipariş bulunamadı."];
        }

        return [
            'status' => 'success',
            'order' => $this->formatOrder($order) + ['customer' => $order->customer->name],
        ];
    }

    private function formatOrder(Order $order): array
    {
        return [
            'order_number' => $order->order_number,
            'order_status' => $order->status,
            'total_amount' => (float) $order->total_amount,
            'notes' => $order->notes,
            'created_at' => $order->created_at?->toDateString(),
        ];
    }
}
```

7.5 ToolSchema.php

app/Schema/ToolSchema.php 47 satır · eksiksiz

```
<?php
namespace App\Schema;

class ToolSchema
{
    /**
     * Anthropic Messages API tool şemaları.
     */
    public static function all(): array
    {
        return [
            [
                'name' => 'get_customer_orders',
                'description' => 'Veritabanında kayıtlı bir müşterinin sipariş geçmişini listeler: sipar
» iş numarası, durum, tutar, not ve tarih. ' .
                                 'Kullanıcı bir müşterinin siparişlerini, sipariş durumlarını veya topla
» m harcamasını sorduğunda bu aracı kullanın. ' .
                                 'Müşteri adı veya soyadıyla arama yapar ve birden fazla müşteri eşleşeb
» ilir (en fazla 5). ' .
                                 'Sipariş numarası ile arama yapmak için bu araç yerine `get_order_statu
» s` kullanın.',
                'input_schema' => [
                    'type' => 'object',
                    'properties' => [
                        'customer_name' => [
                            'type' => 'string',
                            'description' => 'Sorgulanacak müşterinin adı, soyadı veya ikisi birden (Örn
» : Ahmet Yılmaz, Mehmet)'
                        ]
                    ],
                    'required' => ['customer_name']
                ],
            ],
            [
                'name' => 'get_order_status',
                'description' => 'Sipariş numarası vererek tek bir siparişin durumunu (pending, processi
» ng, completed, cancelled), tutarını, notunu, tarihini ve müşterisini getirir. ' .
                                 'Kullanıcı elindeki belirli bir sipariş numarasını sorduğunda bu aracı
» kullanın. ' .
                                 'Müşterinin tüm siparişlerini görmek için bu aracı değil `get_customer_
» orders` kullanın.',
                'input_schema' => [
                    'type' => 'object',
```

```
                    'properties' => [
                        'order_number' => [
                            'type' => 'string',
                            'description' => 'Sipariş numarası, "ORD-" önekiyle (Örn: ORD-1001)'
                        ]
                    ],
                    'required' => ['order_number']
                ],
            ],
        ];
    }
}
```

7.6 Akışa dahil destekleyici dosyalar

Kapsamdaki dosyalar bunlar olmadan anlaşılamayacağı için başvuru amacıyla eklenmiştir.

routes/web.php 14 satır · eksiksiz

```
<?php

use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});


Route::get('/ai/agent', [App\Http\Controllers\Ai\AgentController::class, 'index'])->name('ai.agent');

//ai sende messages

Route::post('/ai/agent/send', [App\Http\Controllers\Ai\AgentController::class, 'post'])->name('ai.send')
;
```

config/services.php yalnızca ilgili bölüm · satır 37–40

```
    'anthropic' => [
        'key' => env('ANTHROPIC_API_KEY'),
        'model' => env('ANTHROPIC_MODEL'), // güncel adı dokümandan al
    ],
```

app/Models/Customer.php 21 satır · eksiksiz

```
<?php

namespace App\Models;

use Database\Factories\CustomerFactory;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Customer extends Model
{
    /** @use HasFactory<CustomerFactory> */
    use HasFactory;

    protected $fillable = ['name', 'email', 'phone', 'city'];

    public function orders(): HasMany
    {
        return $this->hasMany(Order::class);
    }
}
```

app/Models/Order.php 28 satır · eksiksiz

```php
<?php

namespace App\Models;

use Database\Factories\OrderFactory;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Order extends Model
{
    /** @use HasFactory<OrderFactory> */
    use HasFactory;

    protected $fillable = ['customer_id', 'order_number', 'total_amount', 'status', 'notes'];

    protected function casts(): array
    {
        return [
            'total_amount' => 'decimal:2',
        ];
    }

    public function customer(): BelongsTo
    {
        return $this->belongsTo(Customer::class);
    }
}
```

8. Genel Mimari Özeti

8.1 Tek paragrafta sistem

Kullanıcının sorusu `AgentController` üzerinden doğrulanarak `AiAgent`'a verilir. AiAgent, system prompt ve `ToolSchema`'daki tool tanımlarıyla birlikte soruyu `AiClient` aracılığıyla Anthropic Messages API'ye gönderir. Model veri isterse `stop_reason: tool_use` döner. AiAgent ilgili tool'u whitelist korumalı `ToolsService` üzerinden çalıştırır, Eloquent ile veritabanından okunan sonucu `tool_result` olarak geçmişe ekler ve modele tekrar sorar. Bu döngü model nihai cevabı üretene kadar, en fazla 5 tur sürer. Son yanıttaki metin blokları birleştirilip `{reply}` JSON'u olarak tarayıcıya döner. İş seviyesindeki olumsuzluklar modele veri olarak gider; altyapı hataları 502/500'e çevrilir.

8.2 Her dosya tek cümlede

Dosya	Rolü
AgentController	HTTP kapısı: doğrular, ajanı çalıştırır, hatayı/cevabı JSON'a çevirir.
AiAgent	Orkestratör: persona + tool listesi + sınırlı tool-use döngüsü.
AiClient	Anthropic'e tek HTTP çağrısı yapan ince adaptör.
ToolsService	Whitelist korumalı tool uygulamaları; Eloquent ile veri okur, sade dizi döner.
ToolSchema	Modele sunulan tool sözleşmeleri; whitelist'in tek kaynağı.

8.3 Akılda tutulacak sözleşmeler

- **Tool adı = ToolsService metod adı** (snake_case). Biri değişirse diğeri de değişmeli.
- **Tool dönüşü her zaman** `['status' => 'success'|'error', ...]` şeklindedir. `error`, modele `is_error: true` olarak gider.
- **Katmanlar arası veri formatı Anthropic Messages formatıdır** (`messages`, `content` blokları, `stop_reason`).
- **Controller ↔ frontend sözleşmesi:** başarıda `{reply: string}`, hatada `{message: string}`.
- **Yapılandırma:** `ANTHROPIC_API_KEY`, `ANTHROPIC_MODEL` → `config('services.anthropic.*')`.

8.4 Yeni bir tool nasıl eklenir? (reçete)

1. `ToolSchema::all()` dizisine yeni bir tanım ekle: `name`, ayrıntılı `description` (ne zaman kullanılır / kullanılmaz) ve `input_schema`.
2. `ToolsService`'e **aynı isimde** public bir metod ekle: `array $input` alsın, girdiyi normalize etsin, `['status' => …]` dönsün.
3. Gerekirse `AiAgent::$system` prompt'una yeni tool'la ilgili davranış kuralı ekle.
4. `Http::fake` ile bir feature testi yaz (bkz. 6.6).

```
// 1) app/Schema/ToolSchema.php
[
    'name' => 'get_customer_by_email',
    'description' => 'E-posta adresiyle tek bir müşteriyi bulur. İsimle arama için get_customer_orders k
  ullanın.',
    'input_schema' => [
        'type' => 'object',
        'properties' => ['email' => ['type' => 'string', 'description' => 'Örn: ahmet@example.com']],
        'required' => ['email'],
    ],
],

// 2) app/Services/Ai/ToolsService.php
public function get_customer_by_email(array $input): array
{
    $email = strtolower(trim((string) ($input['email'] ?? '')));
    $customer = Customer::where('email', $email)->first();

    return $customer
        ? ['status' => 'success', 'customer' => $customer->only('name', 'email', 'city')]
        : ['status' => 'error', 'message' => 'Müşteri bulunamadı.'];
}
```

8.5 İyileştirme yol haritası

Konu	Öneri	Öncelik
Yetkilendirme	AI route'larına `auth` (ve gerekirse rol kontrolü) ekle.	Yüksek
Rate limiting	`throttle:20,1` gibi bir limit ile maliyet ve kötüye kullanım koruması.	Yüksek
Hata eşleme	`ConnectionException`'ı yakalayıp 502/504 döndür.	Orta
Yapılandırma	`.env.example`'a `ANTHROPIC_API_KEY` ve `ANTHROPIC_MODEL` ekle; boşsa açık hata ver.	Orta
Dayanıklılık	`Http::retry()` ile 429/529 için geri çekilmeli tekrar deneme.	Orta
Testler	`Http::fake` ile ajan döngüsü, whitelist ve hata dalları için feature testleri.	Orta
Sohbet geçmişi	Frontend'in geçmişi göndermesi veya oturumda saklama; `messages` dizisine ekleme.	Düşük
Maliyet	System prompt ve tools için prompt caching; `usage` alanını loglama.	Düşük
UX	`max_tokens` kesilmesini algılayıp kullanıcıyı bilgilendirme; streaming (SSE).	Düşük
Doğruluk	LIKE sorgusunda `%` ve `_` karakterlerini escape etme.	Düşük

8.6 Altı ay sonra geri döndüğünüzde kontrol listesi

- İstek nereden başlıyor? → `routes/web.php` → `AgentController::post`.
- Model ne biliyor? → Yalnızca `AiAgent::$system` + `ToolSchema::all()` + o anki `$messages`.
- Veri nereden geliyor? → Yalnızca `ToolsService` metotları (Eloquent).
- API ile ilgili bir sorun mu var? → `storage/logs/laravel.log` içinde “Anthropic API hatası” kaydı (status + body).
- Model yanlış tool seçiyor mu? → Önce `ToolSchema` açıklamalarını iyileştir, kodu değil.
- Model veri uyduruyor mu? → System prompt kurallarını ve tool çıktısının yeterliliğini kontrol et.